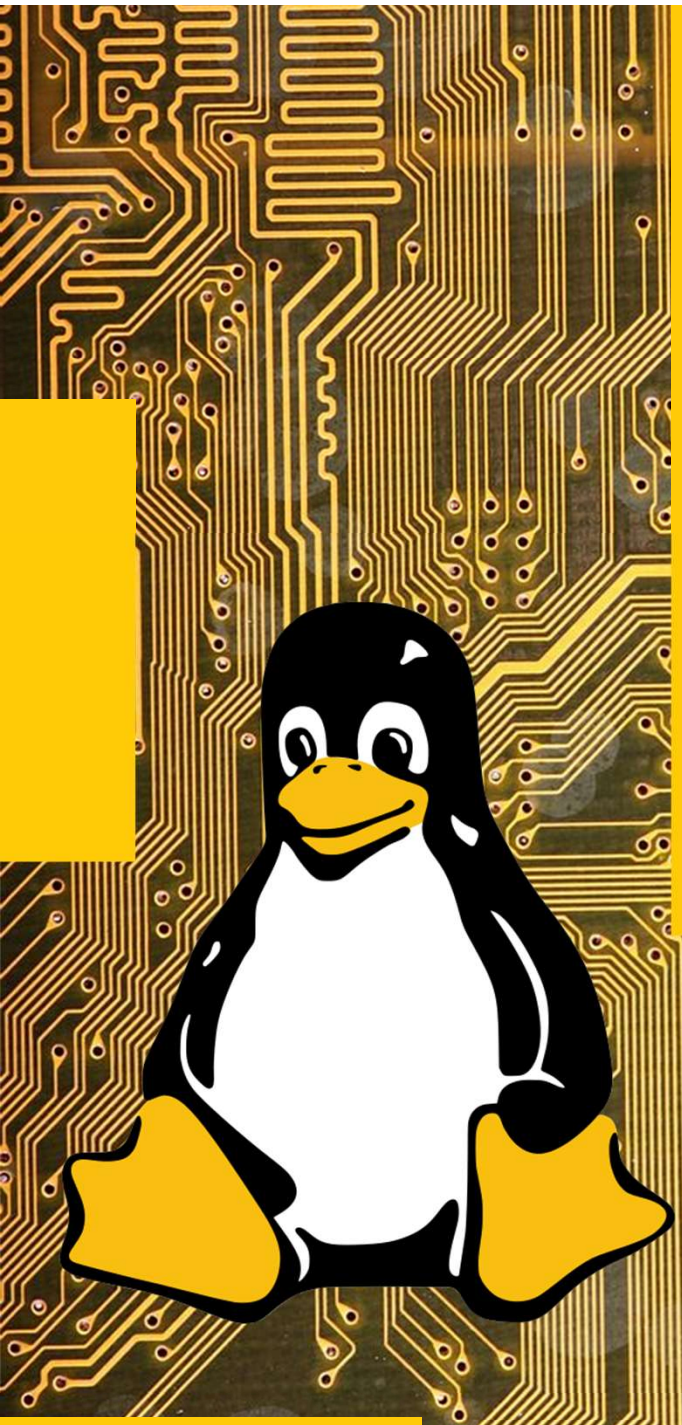


INHA UNIVERSITY IN TASHKENT

SOC 3010 OPERATING SYSTEMS

CPU Scheduling

INSTRUCTOR: Jalolliddin Yusupov



The Problem the Kernel Must Solve

A process runs **on the real CPU**, with the real registers. That is why it is fast — and why it is dangerous: while it runs, **the kernel is not running**.

So who gives the CPU back? Two answers in history:

Cooperative	The program hands the CPU over itself: it calls `yield()` (a system call whose only job is "let someone else run") or any other system call. Windows 3.x and classic Mac OS worked this way. One `while (1);` in any program froze the whole machine — that was the 1990s.
Preemptive	The kernel takes the CPU *away*. The tool is the **timer interrupt**: a hardware clock raises an interrupt every few milliseconds; the CPU stops the process mid-instruction, jumps into the kernel, and the kernel decides who runs next.

Two names

Interrupt — a hardware signal that makes the CPU stop what it is doing and run a kernel routine (the *handler*) — the same idea as a signal handler, one floor down.

Preemption — taking the CPU from a process that did not ask to give it up. The process never notices; it wakes up later with its registers (context switch) exactly as they were.

What the Scheduler Decides, and for Whom

Only **runnable** processes are on it: state **R** in `ps`. Sleeping ones are off it. **S** is waiting for something interruptible: a pipe, a signal, a `sleep()`. **D** is waiting in the kernel for a disk or network operation that *cannot* be interrupted, not even by `kill -9`.

- **Which** runnable process gets the CPU next.
- **For how long** before the timer takes it back.

Two customers who want opposite things

Why this is a fight

Every millisecond the CPU spends switching to the editor is a millisecond the encode did not get. There is no setting that makes both happy; there is only a choice of whom to disappoint, and by how much.

Throughput

A compiler, a video encode, a database rebuild. Wants throughput: jobs finished per hour. Does not care whether it waited 3 s to start.

Latency

There is no fair; there is only a chosen victim.

Your editor, a shell, a game. Wants latency: the time between a key press and the screen reacting, ideally under ~50 ms. Does not care how long it takes to finish, because it never finishes.

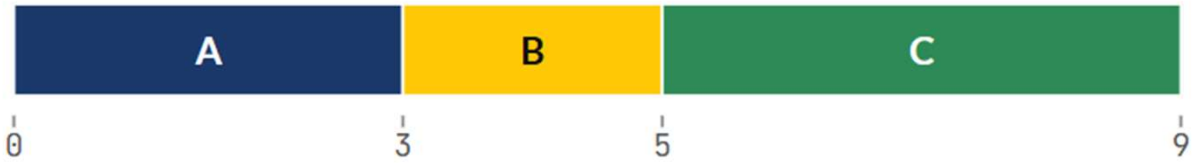
The Metrics, and the Lies We Start With

THREE NUMBERS PER JOB

Turnaround	$T_{completion} - T_{arrival}$. How long from "job exists" to "job finished". The batch customer's number.
Response	$T_{first\ run} - T_{arrival}$. How long until it did *anything*. The interactive customer's number.
Fairness	Does everyone get a share? Is anyone waiting forever?

GANTT CHART (THE EXAM METHOD)

A GANTT CHART: WHO HOLDS THE CPU, WHEN



A time line: one box per stretch of CPU ownership. Read completion times at the right edges, first-run times at the left edges. Here A: turnaround 3, response 0 · B: 5, 3 · C: 9, 5.

Waiting time – amount of time a process has been waiting in the ready queue

Four assumptions we will drop one by one

1.Every job runs for the same time.
2.All jobs arrive at the same moment.
3.Once started, a job runs to completion — no I/O.
4.The run time of each job is **known**.

Unrealistic, all four. They make the first policy look fine.

FIFO and the Convoy Effect

Burst time	the total CPU time the process needs

CPU burst	A continuous period during which a process uses the CPU.

Process	Burst Time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
  The Gantt Chart for the schedule is:



- Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
- Average waiting time: (0 + 24 + 27)/3 = 17

FIFO and the Convoy Effect

Suppose that the processes arrive in the order:

$$P_2 , P_3 , P_1$$

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6; P_2 = 0; P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3
- Much better than previous case
- **Convoy effect** - short process behind long process
  - hurting average waiting time and response time

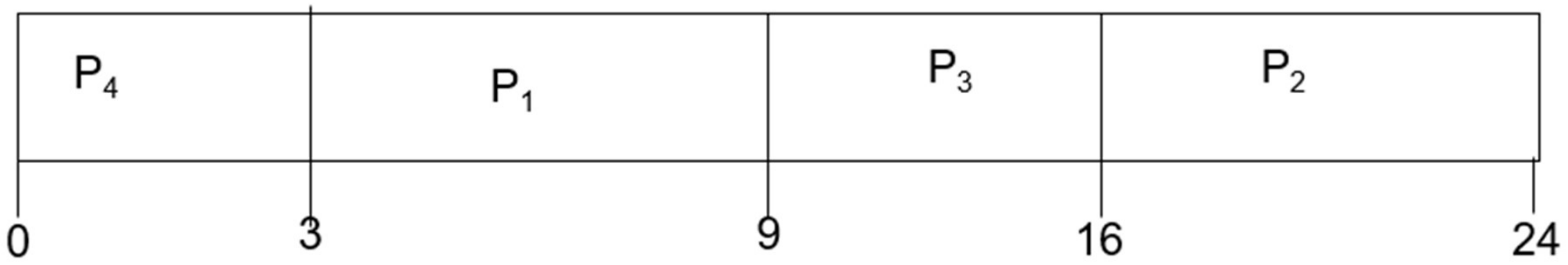
Shortest-Job-First (SJF) [non-peemptive]

- Associate with each process the length of its next CPU burst
  - Use these lengths to schedule the process with the shortest time

- SJF is optimal – gives minimum average waiting time for a given set of processes
  - The **difficulty** is knowing the length of the **next CPU request**
  - Could ask the user

Process	Burst Time
P_1	6
P_2	8
P_3	7
P_4	3

Average waiting time:
=(3 + 16 + 9 + 0) / 4 = 7



(SRTF) Shortest-remaining-time-first [preemptive]

- Now we add the concepts of varying arrival times and preemption to the analysis

Process	*Arrival* Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

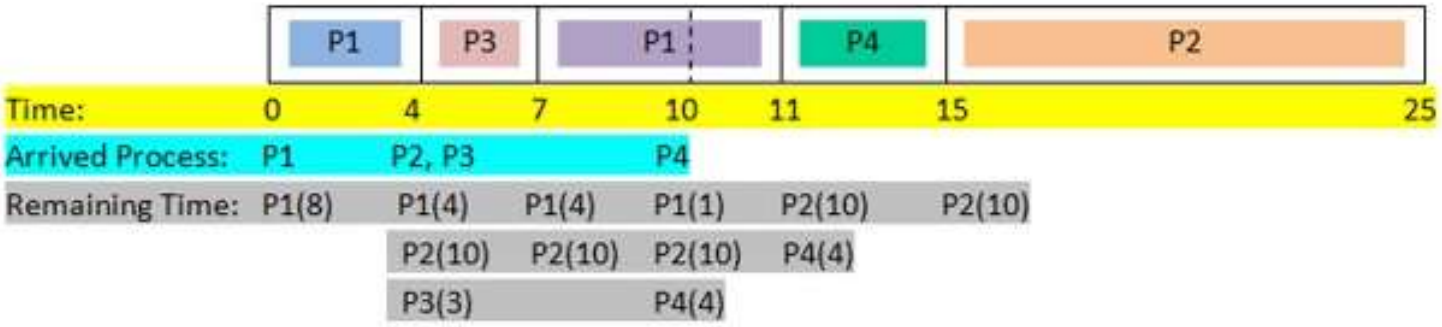
- *Preemptive* SJF Gantt Chart



- Average waiting time = [(10-1)+(1-1)+(17-2)+5-3)]/4 = 26/4 = 6.5 msec

(SRTF) Shortest-remaining-time-first [preemptive]

Process	Arrival Time	CPU Burst Time
P1	0	8
P2	4	10
P3	4	3
P4	10	4



Average Turnaround Time =Sum of Turnaround Time of each Process/ Number of Processes

$$= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4$$

$$= ((11 - 0) + (25 - 4) + (7 - 4) + (15 - 10)) / 4 = 10 \text{ ms}$$

Average Waiting Time = Sum of Waiting Time of Each Process/ Number of processes

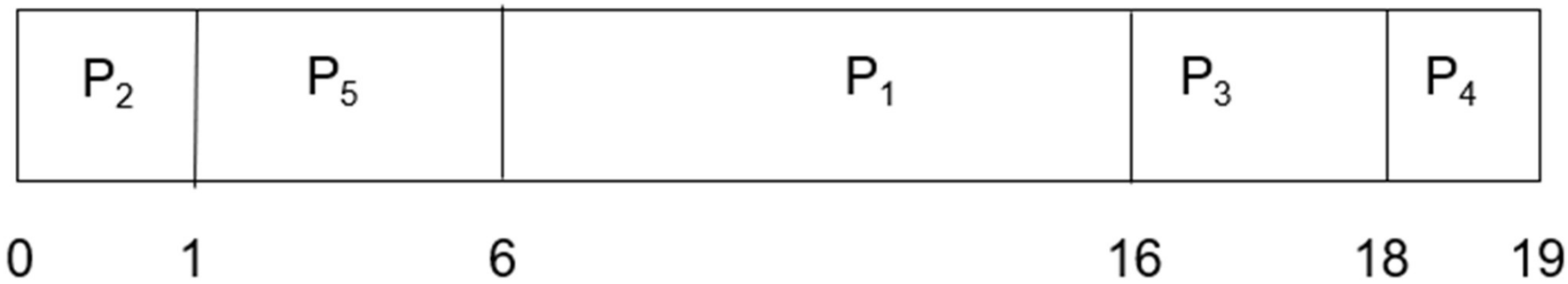
$$= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4$$

$$= (3 + 11 + 0 + 1) / 4 = 3.75 \text{ ms}$$

Priority Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

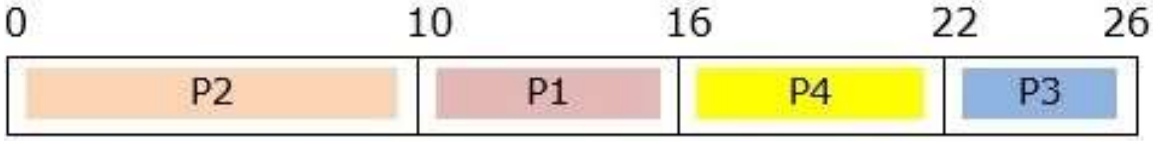
- Priority scheduling Gantt Chart



- Average waiting time = 8.2 msec
- low priority can starve
  - aging fixes it

(Example) Priority Scheduling [non-Preemptive]

Process	CPU Burst Time in ms	Priority
P1	6	2
P2	10	1
P3	4	3
P4	6	2



Average Turnaround Time

= Sum of Turnaround Time of each Process/ Number of Processes

= (TAT_{P1} + TAT_{P2} + TAT_{P3} + TAT_{P4}) / 4

= (16 + 10 + 26 + 22) / 4 = 18.5 ms

Average Waiting Time

= Sum of Waiting Time of Each Process/ Number of processes

= (WT_{P1} + WT_{P2} + WT_{P3} + WT_{P4}) / 4

= (10 + 0 + 22 + 16) / 4 = 10.5 ms

(Homework) Priority Scheduling [preemptive]

Process	Arrival Time	CPU Burst Time	Priority
P1	0	8	3
P2	4	10	2
P3	4	3	3
P4	10	4	1

Average Turnaround Time

Average Waiting Time



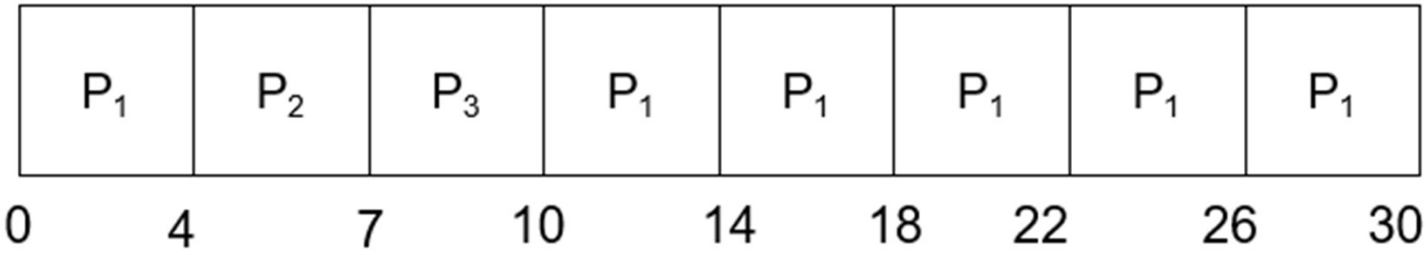
Time:	0	4	10	14	18	22	25
Arrived Process:	P1	P2, P3	P4				
Remaining Time:	P1(8)	P1(4) P2(10) P3(3)	P1(4) P2(4) P3(3) P4(4)	P1(4) P2(4) P3(3)	P1(4) P3(3)	P3(3)	

Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum** *q*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are *n* processes in the ready queue and the time quantum is *q*, then each process gets 1/*n* of the CPU time in chunks of at most *q* time units at once. No process waits more than (*n*-1)*q* time units.
- Timer interrupts every quantum to schedule next process
- Performance
  - *q* large ⇒ FIFO
  - *q* small ⇒ *q* must be large with respect to context switch, otherwise overhead is too high

Process	Burst Time
P_1	24
P_2	3
P_3	3

(**time quantum** *4*)



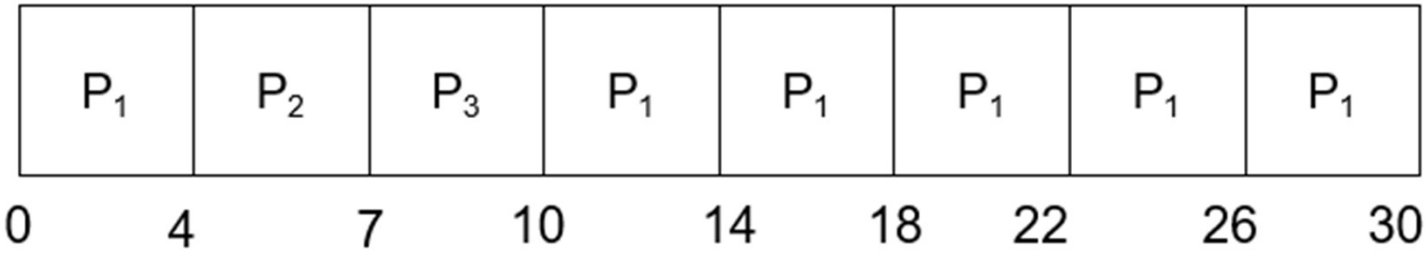
Example of RR with Time Quantum = 4

- Typically, higher average turnaround than SJF, but better ***response***
- q should be large compared to **context switch** time
- q usually 10ms to 100ms, context switch < 10 usec

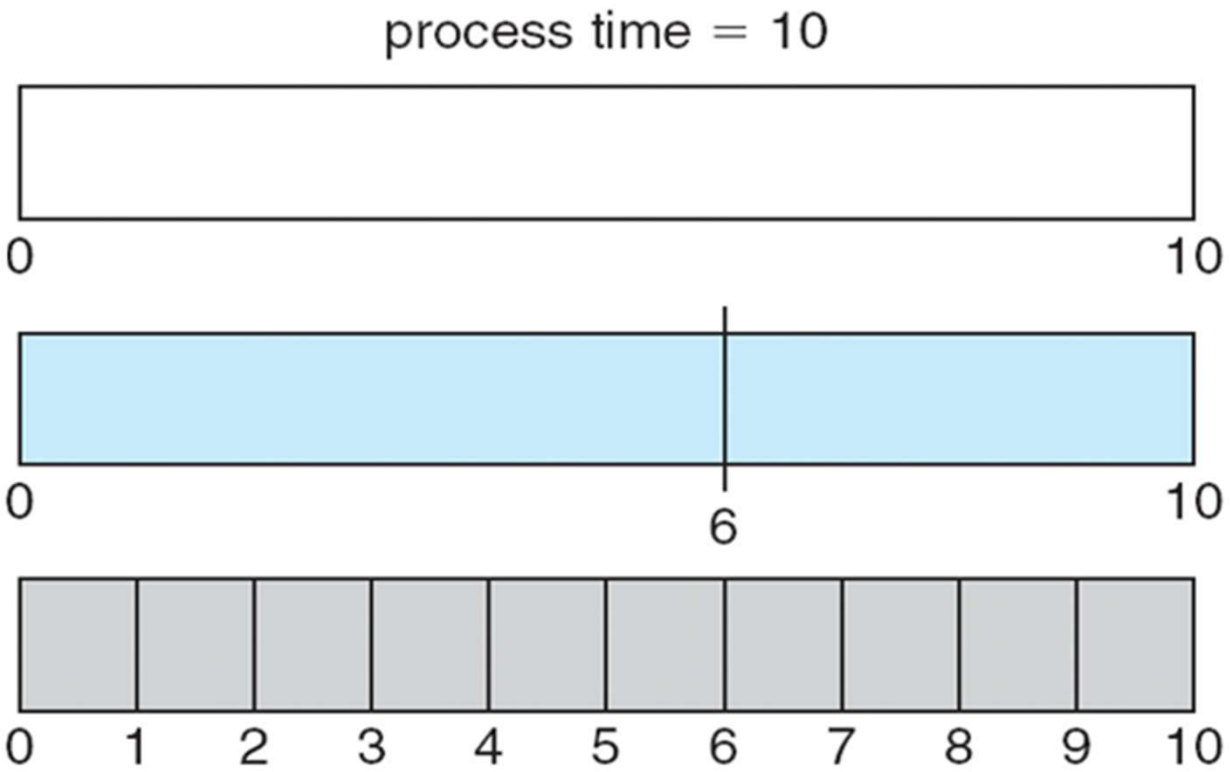
A **context switch** is the kernel saving one process's registers and stack pointer into that process's kernel record, restoring the next one's, and jumping. The interrupted process does not know it happened.

Process	Burst Time
P_1	24
P_2	3
P_3	3

(**time quantum** *4*)



Time Quantum and Context Switch Time



quantum	context switches
12	0
6	1
1	9

Round Robin (RR) (Homework)

Process	CPU Burst Times in ms
P1	8
P2	10
P3	6
P4	4

Average Turnaround Time

Average TAT = Sum of Turnaround Time of each Process / Number of Processes

$$= (\mathrm{TAT_{P1}} + \mathrm{TAT_{P2}} + \mathrm{TAT_{P3}} + \mathrm{TAT_{P4}})/4$$

= (24 + 28 + 22+ 16) / 4 = 22.5 ms

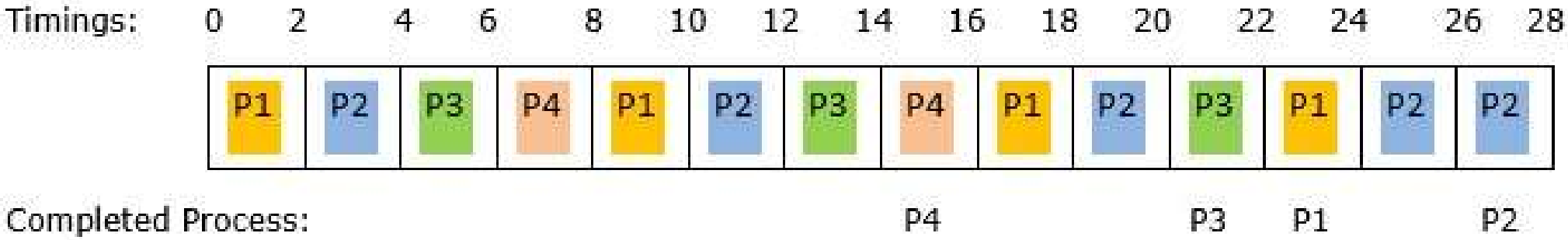
In order to calculate the waiting time of each process, we multiply the time quantum with the number of time slices the process was waiting in the ready queue.

Average Waiting Time

Average WT = Sum of Waiting Time of Each Process / Number of processes

$$= (\mathrm{WT_{P1}} + \mathrm{WT_{P2}} + \mathrm{WT_{P3}} + \mathrm{WT_{P4}})/4$$

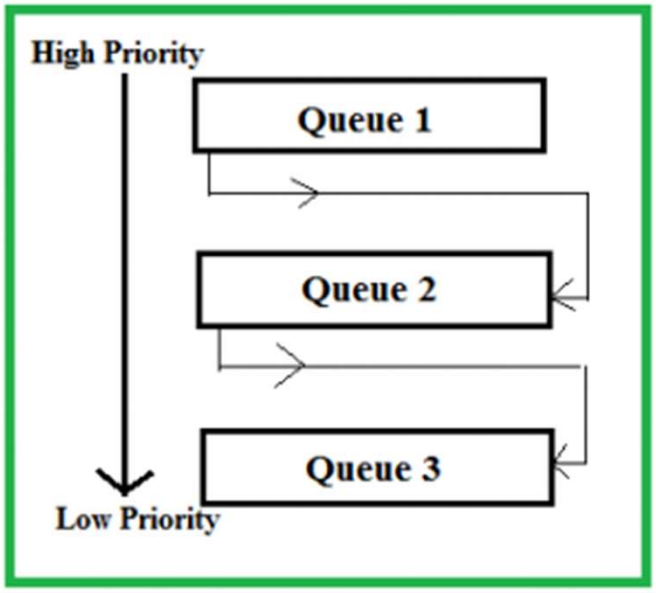
= (8*2 + 9*2+ 8*2+ 6*2) / 4 = 15.5 ms



MLFQ: Learning the Future from the Past

SJF is optimal and impossible: nobody knows a job's length in advance.
The **multi-level feedback queue** (MLFQ) guesses it from the job's *past*.

Floor	Rule	Who goes here?
Queue 0 (Top floor)	Time Quantum = **1**	Brand new processes
Queue 1 (Middle floor)	Time Quantum = **2**	Processes that didn't finish in Q0
Queue 2 (Ground floor)	**FCFS** (no limit)	Processes that didn't finish in Q1



- **Key Rule:** Always run the **highest queue first**. Only go to a lower queue when the higher one is **empty**.
- If a process **finishes** within its time quantum → ✅ Great! It's done.
- If a process **doesn't finish** → ⬇️ It gets **demoted** (pushed down) to the next queue.
- This is the **"feedback"** — the system learns: *"This process needs more time, let's give it a bigger slice but lower priority."*

MLFQ is shortest-job-first for people who cannot see the future — and every desktop OS is a tuned version of it.



Questions?